

UNIT 5

✓ 1 Mark Questions and Answers

1. What is a class in Python?

→ A class is a blueprint for creating objects, bundling attributes (variables) and methods (functions).

2. What is the purpose of the `__init__()` method in a class?

→ It is a constructor used to initialize object attributes when an object is created.

3. What does the `self` keyword represent?

→ `self` represents the current instance of the class.

4. Write the syntax to create an object of a class.

A) `obj = ClassName()`

5. Differentiate between class attribute and instance attribute.

→ Class attribute is shared by all objects of a class, whereas instance attribute is unique to each object.

6. Can a class have multiple constructors in Python? (Yes/No)

→ No, Python does not support multiple constructors directly.

7. Which keyword is used to define a class in Python?

→ `class`

8. What is the default return type of a method if `return` is not used?

→ `None`

Questions from Inheritance:

9. What is inheritance in Python?

→ Inheritance allows a class (child) to acquire properties and methods from another class (parent).

10. Which keyword is used for inheritance in Python?

→ `class Child(Parent):`

11. What type of inheritance allows a class to inherit from more than one parent?

→ Multiple Inheritance.

12. What is method overriding?

→ When a child class defines a method with the same name as the parent class, it overrides the parent's method.

13. Does Python support method overloading? (Yes/No)

→ No, but it can be achieved using default arguments or variable-length arguments.

14. What is the base class of all classes in Python?
→ object

Encapsulation

1. What is encapsulation in Python?
→ Encapsulation is the process of restricting direct access to class variables and methods.
2. Which symbols are used for private and protected attributes in Python?
→ `_var` for protected and `__var` for private.
3. Can private variables be accessed directly outside a class? (Yes/No)
→ No.
4. What are getter and setter methods?
→ Special methods used to access (get) and update (set) private attributes.

Polymorphism

5. What is polymorphism in Python?
→ The ability of a function/method to take on many forms (same name, different behavior).
 6. Which OOP principle allows method overriding?
→ Polymorphism.
 7. Give an example of a built-in polymorphic function in Python.
→ `len()` works for strings, lists, tuples, etc.
-

2 Mark Questions and Answers

Types of Inheritance

Type Description

Single One parent → One child

Multiple Many parents → One child

Multilevel Chain of inheritance

Hierarchical One parent → Many children

a) Single Inheritance

Parent
↓
Child

b) Multiple Inheritance

Parent1 Parent2
 \ /
 \ /
 Child

c) Multilevel Inheritance

Grandparent
↓
Parent
↓
Child

d) Hierarchical inheritance

Parent
/ \
Child 1 Child 2

1. Explain the difference between a class and an object with an example.

→ A class is a blueprint, and an object is an instance of the class.

class Student: # Student is a class

def __init__(self, name, age):

self.name = name

self.age = age

def display(self):

print(self.name, self.age)

s1 = Student("Ram", 19) # s1 is an object

2. Write a simple Python class with attributes.

class Student: # Student is a class

def __init__(self, name, age): # name and age are attributes

self.name = name

self.age = age

def display(self):

print(self.name, self.age)

3. How is self used inside a class? Give an example.

→ It is used to access instance variables and methods.

class Student:

def __init__(self, name):

self.name = name

4. Define a class Student with a constructor that initializes name and marks.

class Student:

def __init__(self, name, marks):

self.name = name

self.marks = marks

5. Explain the role of methods in classes with an example.

→ Methods define the behavior of objects.

class Calculator:

def add(self, a, b): # method

return a + b

Questions from Inheritance:

6. Explain single inheritance with an example.

→ A child inherits from one parent class.

```
1 class Parent():
2     def show_p(self):
3         print("I am Parent")
4
5 class Child(Parent):
6     def show_c(self):
7         print("I am Child")
8
9 c = Child() # Object creation with child class
10 c.show_c()
11 c.show_p()
```

```
I am Child
I am Parent
```

7. Differentiate between single and multiple inheritance.

- **Single Inheritance** → Child inherits from one parent.
- **Multiple Inheritance** → Child inherits from two or more parents.

8. What is multilevel inheritance? Give an example.

→ In multilevel inheritance, a class is derived from another derived class.

```
class A: pass
class B(A): pass
class C(B): pass
```

9. Explain hierarchical inheritance.

→ Multiple child classes inherit from the same parent class.

10. What is the advantage of inheritance?

→ Code reusability and extensibility.

Encapsulation

1. Differentiate between public, protected, and private members in Python.

- **Public:** Accessible anywhere.
- **Protected (`_var`):** Accessible within class and subclasses.
- **Private (`__var`):** Accessible only within the class.

2. Write a simple program to demonstrate private variables.

```
class Student:
    def __init__(self, name):
        self.__name = name # private variable
```

```
s = Student(" John")
print(s._Student__name)
```

Output: John

Polymorphism

3. Differentiate between method overloading and method overriding.

- Overloading: Same method name, different parameters (not directly supported in Python).
- Overriding: Child class redefines parent class method.

4. Write an example of polymorphism using a common method.

```
class Manager:
    def salary(self):
        print("Manager salary: 80000")

class Developer:
    def salary(self):
        print("Developer salary: 60000")

class Intern:
    def salary(self):
        print("Intern stipend: 15000")
```

```
# Polymorphism
Manager().salary()
Developer().salary()
Intern().salary()
```

Output:

```
Manager salary: 80000
Developer salary: 60000
Intern stipend: 15000
```

5. Explain the difference between encapsulation and polymorphism.

- **Encapsulation** → Bundles data and methods, restricts access.
Encapsulation is about *hiding and protecting data*. It means bundling variables (data) and methods (functions) inside a class, and restricting direct access to some of the object's components. Instead of accessing data directly, you use controlled methods (getters/setters).
- **Polymorphism** → Same function name, different behavior.
Example: **len("abc")** → 3, **len([1, 2, 3])** → 3.
Polymorphism is about *one interface, many forms*. It allows the same method or function to behave differently depending on the object or context.

5 Mark Questions and Answers

1. Explain the concept of class and object in Python with an example program.

→ A class is a blueprint; objects are instances. Example:

```
class Student: # Student is a class
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def display(self):
```

```
        print(self.name, self.age)
```

```
s1 = Student("Ram", 19) # s1 is an object
```

```
s1.display()
```

Output: Ram, 19

2. Write a Python program using a class **Employee** with attributes **name** and **salary**. Include a method to display employee details.

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self.salary = salary
```

```
    def display(self):
```

```
        print(f"Name: {self.name}, Salary: {self.salary}")
```

```
e = Employee("Sam", 50000) # Object creation
```

```
e.display() # Method calling
```

Output: Name: Sam, Salary: 50000

3. Write a program using a class **Calculator** to perform addition, subtraction, multiplication, and division.

```
class Calculator:
```

```
    def add(self, a, b): return a + b
```

```
    def sub(self, a, b): return a - b
```

```
    def mul(self, a, b): return a * b
```

```
    def div(self, a, b): return a / b if b != 0 else "Error: Division by zero"
```

```
c = Calculator()
```

```
print(c.add(10, 5))
```

```
print(c.sub(10, 5))
```

```
print(c.mul(10, 5))
```

```
print(c.div(10, 2))
```

Output:

15
5
50
5

4. Discuss the difference between class variables and instance variables with examples.

- **Class variable** → Shared among all objects.
- **Instance variable** → Unique to each object.

```
class Student:
    college = "MGIT"      # Class variable
    def __init__(self, name):
        self.name = name  # Instance variable
```

```
s1 = Student("John")
s2 = Student("Sam")
print(s1.college, s1.name)
print(s2.college, s2.name)
```

Output: MGIT John
MGIT Sam

5. Create a class **Book** with attributes **title**, **author**, and **price**. Write methods to display the book details and check if the book is expensive (price > 500).

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def display(self):
        print("Title:", self.title)
        print("Author:", self.author)
        print("Price:", self.price)

    def is_expensive(self):
        if self.price > 500:
            print("The book is expensive")
        else:
            print("The book is affordable")

b1 = Book("Python Basics", "Ravi", 650)
b1.display()
b1.is_expensive()
```

Output: Title: Python Basics, Guido, 650
The book is expensive

Questions from Inheritance:

6. Explain the different types of inheritance in Python with examples.

- **Single Inheritance** – One parent, one child.
- **Multilevel Inheritance** – Inheritance across multiple levels.
- **Multiple Inheritance** – Child inherits from many parents.
- **Hierarchical Inheritance** – Many children inherit from one parent.

1. Eg of Single inheritance:

```
1 class Person: # Parent class
2     def basic_info(self):
3         print("Learning things to find a job in a company")
4
5 class Employee(Person): # Child class
6     def role(self):
7         print("Attend works assigned by the company")
8
9 # Create object of child class
10 m = Employee()
11
12 m.basic_info() # inherited method
13 m.role() # child class method
```

Employee Name: Ravi
Role: Manager

2. Example of Multilevel Inheritance:

```
1 class Person:
2     def basic_info(self):
3         print("Learning things to find a job in a company")
4
5 class Employee(Person):
6     def role(self):
7         print("Attend works assigned by the company")
8
9 class Manager(Employee):
10     def decision(self):
11         print("Manager makes decisions")
12
13 # Create object
14 s = Manager()
15
16 s.basic_info() # from Person
17 s.role() # from Employee
18 s.decision() # from Manager or own method
```

Learning things to find a job in a company
Attend works assigned by the company
Manager makes decisions

3. Example of Multiple Inheritance:

```
1 class Father:
2     def skill1(self):
3         print("Father: Gardening")
4
5 class Mother:
6     def skill2(self):
7         print("Mother: Cooking")
8
9 class Child(Father, Mother):    # Inheriting from both
10     def skill3(self):
11         print("Child: Playing")
12
13 # Create object
14 c = Child()
15
16 c.skill1()    # from Father
17 c.skill2()    # from Mother
18 c.skill3()    # own method
```

Father: Gardening
Mother: Cooking
Child: Playing

4. Example of Hierarchical Inheritance:

```
1 # Parent class
2 class Employee:
3     def display_company(self):
4         print("Company: TCS")
5
6 # Child class 1
7 class Manager(Employee):
8     def manager_role(self):
9         print("Role: Manager")
10
11 # Child class 2
12 class Developer(Employee):
13     def developer_role(self):
14         print("Role: Developer")
15
16 # Creating objects
17 m = Manager()
18 d = Developer()
19
20 # Accessing methods
21 m.display_company()
22 m.manager_role()
23
24 d.display_company()
25 d.developer_role()
```

Company: TCS
Role: Manager
Company: TCS
Role: Developer

7. Explain encapsulation and polymorphism in relation to inheritance.

- Encapsulation means **wrapping data (variables) and methods together** and **restricting direct access** to data.
- In inheritance, **encapsulation controls access** to data.

```
1 class Student:
2     def __init__(self, marks, grade):
3         self._marks = marks # protected
4         self.__grade = grade # private
5
6 s = Student(95, 'A+')
7 print(s._marks) # to access protected variable
8 print(s._Student__grade) # to access private variable
```

95

A+

- **Polymorphism** → Same function name behaves differently in different classes.
- In inheritance, polymorphism is achieved by **overriding methods**.

```
1 class Person: # Parent class
2     def display(self):
3         print("Learning things to find a job in a company")
4
5 class Employee(Person): # Child class
6     def display(self):
7         Person.display(self)
8         # super().display()
9         print("Attend works assigned by the company")
10
11 # Create object of child class
12 m = Employee()
13
14 m.display() # calling overridden method
```

Learning things to find a job in a company

Attend works assigned by the company

8. Write a program using multilevel inheritance where **Person** → **Employee** → **Manager**.

```

1 class Person:
2     def basic_info(self):
3         print("Learning things to find a job in a company")
4
5 class Employee(Person):
6     def role(self):
7         print("Attend works assigned by the company")
8
9 class Manager(Employee):
10    def decision(self):
11        print("Manager makes decisions")
12
13 # Create object
14 s = Manager()
15
16 s.basic_info()    # from Person
17 s.role()          # from Employee
18 s.decision()      # from Manager or own method

```

Learning things to find a job in a company
 Attend works assigned by the company
 Manager makes decisions

Polymorphism

9. Write a program to demonstrate polymorphism using method overriding.

Sol: **super().method** can be used to access the methods from the parent class in child class. Or parent class method's can be called **Parent.method(self)**.

```

1 class Person: # Parent class
2     def display(self):
3         print("Learning things to find a job in a company")
4
5 class Employee(Person): # Child class
6     def display(self):
7         Person.display(self)
8     # super().display()
9     print("Attend works assigned by the company")
10
11 # Create object of child class
12 m = Employee()
13
14 m.display()    # calling overridden method

```

Learning things to find a job in a company
 Attend works assigned by the company

10. What are the types of polymorphism? Discuss how Python achieves polymorphism without method overloading. Give an example.

Sol: Types of polymorphism: a) Method overriding and b) Method overloading

a) Python achieves polymorphism by using super() method to overcome “method overriding”.

b) Python achieves polymorphism without “method overloading” by using default arguments or *args.

class Calculator:

def add(self, a, b=0, c=0):

return a + b + c

m = Calculator()

print(m.add(5)) **# 1 argument**

print(m.add(5, 10)) **# 2 arguments**

print(m.add(5, 10, 15)) **# 3 arguments**

Output:

5

15

30

11. What are the types of Constructors? Explain with examples.

A) Parameterized constructor: Constructor with parameters is known as parameterized constructor. The parameterized constructor takes its first argument as a reference to the instance being constructed known as self and the rest of the arguments are provided by the programmer. **Takes parameters to initialize object with specific values.**


```

1 class Calculator:    # Class
2     def __init__(self, a, b):    # Parametrized constructor
3         self.a = a
4         self.b = b
5     def add(self):    # Methods: add, sub
6         return self.a + self.b
7     def sub(self):    # Attributes: a, b
8         return self.a - self.b
9
10 calc = Calculator(10,5)    # Object1 creation with parameters
11 print(calc.add())    # calling method with no parameters
12 print(calc.sub())
13
14 calc2 = Calculator(100,40)    # Object2 creation
15 print(calc2.add())
16 print(calc2.sub())

```

15
5
140
60

Default constructor: The default constructor is a simple constructor which doesn't accept any arguments. Its definition has only one argument which is (self) reference to the instance being constructed.

- **Does not take any parameters (except self).**
- **Initializes object with default values.**

1 Eg of Default Constructor:

```

1 class Calculator:    # Class
2     def __init__(self):    # Default parameters
3         self.a = 10
4         self.b = 5
5     def add(self):    # Methods: add, sub
6         return self.a + self.b
7     def sub(self):    # Attributes: a, b
8         return self.a - self.b
9
10 calc = Calculator()    # Object1 creation with default parameters
11 print(calc.add())
12 print(calc.sub())

```

15
5